🧩 1. What is Encapsulation?

Encapsulation is the process of **wrapping data (variables)** and **code (methods)** into a single unit — that is, a **class**.

It also means **restricting direct access** to an object's data to protect it from accidental modification.

In short:

"Encapsulation = Data Hiding + Data Protection"

🧱 2. Why Encapsulation?

Imagine you have a **Bank Account** object.
You don't want anyone to directly change the **balance** value from outside!

❌ Wrong way:

```
account.balance = -5000   # Dangerous! Invalid balance
```

✅ Right way:
Use **methods** to control how the balance is accessed and changed.

🧠 3. Access Modifiers in Python

Python doesn't have strict access modifiers like other languages (e.g., private, public in Java). Instead, it uses **naming conventions**:

Modifier	Syntax	Access	Example
Public	variable	Accessible from anywhere	self.name
Protected	_variable	Accessible within class and subclasses	self._age
Private	__variable	Hidden from outside access	self.__balance

💼 4. Practical Example: Bank Account

Let's create a secure BankAccount class.

```
class BankAccount:
```

```
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder    # public
        self.__balance = balance                # private

    # Method to deposit
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited ₹{amount}")
        else:
            print("Deposit amount must be positive.")

    # Method to withdraw
    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrew ₹{amount}")
        else:
            print("Invalid withdrawal amount.")

    # Getter method (to read balance safely)
    def get_balance(self):
        return self.__balance
```

🚀 **5. Using the Class**

```
acc = BankAccount("John", 5000)
```

```
acc.deposit(2000)

acc.withdraw(1000)

print("Balance:", acc.get_balance())


# Trying to access private attribute directly

print(acc.__balance)   # ❌ Error: AttributeError
```

📜 **Output:**

```
Deposited ₹2000

Withdrew ₹1000

Balance: 6000

AttributeError: 'BankAccount' object has no attribute '__balance'
```

✅ The private variable __balance is **hidden** and only accessible through methods.

🔓 6. Accessing Private Data (Not Recommended)

You *can* technically access it using *name mangling*,
but this is **not good practice**.

```
print(acc._BankAccount__balance)  # ⚠️ Works, but avoid this
```

🧩 7. Getters and Setters

We use these to **get** and **set** private data safely.

Example:

```
class Student:

    def __init__(self, name, grade):

        self.__name = name

        self.__grade = grade
```

```
    # Getter
    def get_grade(self):
        return self.__grade

    # Setter
    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
        else:
            print("Invalid grade!")

stu = Student("Alice", 85)
print(stu.get_grade())  # 85
stu.set_grade(92)
print(stu.get_grade())  # 92
stu.set_grade(200)      # Invalid grade!
```

🧠 *Setters help you validate data before saving it.*

🧪 8. Practice Exercise

Try this yourself 👇
(No answers, just instructions)

```
# Exercise: Encapsulation Practice
# 1. Create a class 'Employee'
#    - private attributes: __name, __salary
# 2. Add methods:
```

- set_name() and get_name()

- set_salary() (salary should be > 0) and get_salary()

3. Create an object and test your methods

📜 9. Summary

Concept	Meaning	Example
Encapsulation	Wrapping data & methods	Class
Public	Accessible everywhere	self.name
Protected	Accessible within class/subclass	self._age
Private	Hidden from outside	self.__balance
Getter/Setter	Safe data access	get_balance() / set_balance()

🧠 Real-Life Analogy

Real World	Code Equivalent
ATM Machine	Class
ATM Display	Getter (read only)
PIN Entry	Setter (controlled access)
Cash balance	Private variable